

Universidad Autónoma de Chiriquí

Facultad de Economía

Escuela De Ciencias de Computación



Lic. en Gestión de Tecnología de la Información

¨Programación 5 PROG 309¨

Integrante:

Daniela Isabel Pittí Miranda

Prof. Alexis Flores

Trabajo Escrito

Laboratorio 1

Fecha de Entrega

27/08/2026

II Semestre

1. OBJETIVO DEL LABORATORIO

1.1 Objetivo General

Configurar el entorno de desarrollo que se utilizará durante el semestre y realizar un primer acercamiento visual al patrón de capas.

1.2 Objetivos Específicos

- Instalar y configurar el JDK 17 y Eclipse IDE para el desarrollo en Java durante el semestre.
- Crear un proyecto Maven base con estructura de paquetes estilo MVC (modelo, vista, controlador).
- Instalar y configurar Git como sistema de control de versiones.
- Crear y preparar un repositorio de trabajo personal (local y remoto en GitHub) para las prácticas del semestre.
- Reconocer visualmente las capas de una aplicación cotidiana y representarlas en un diagrama.
- Documentar el procedimiento realizado mediante un informe técnico de laboratorio.

2. MARCO TEÓRICO

2.1 El Entorno de Desarrollo Integrado (IDE)

Un IDE (Integrated Development Environment) es una aplicación que reúne en un solo lugar el editor de código, el depurador (debugger), el compilador o intérprete, y herramientas de gestión de proyectos. Ejemplos ampliamente usados en la industria son Visual Studio Code, IntelliJ IDEA y Eclipse.

En este laboratorio se utilizó **Eclipse IDE for Enterprise Java and Web Developers** como entorno estándar. Eclipse ofrece integración nativa con Maven, con servidores de aplicaciones (Tomcat) y con Git, lo cual será especialmente útil en los ejes temáticos de patrón de capas y patrón cliente-servidor.

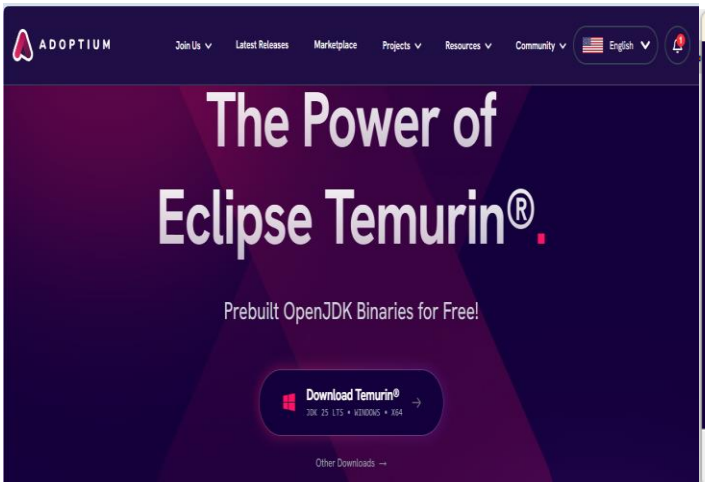
2.2 Control de Versiones con Git

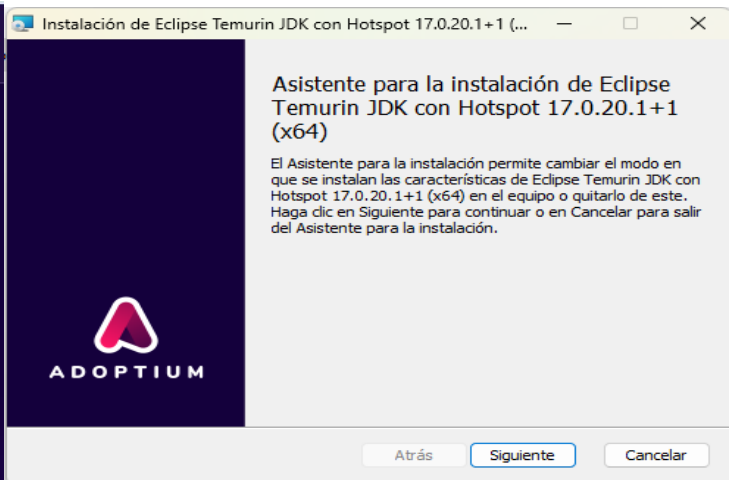
Git es un sistema de control de versiones distribuido que permite registrar el historial de cambios de un proyecto, trabajar de forma colaborativa y recuperar versiones anteriores del código cuando sea necesario. Un repositorio (repo) es el espacio donde Git almacena ese historial; puede alojarse de forma local o remota, por ejemplo, en GitHub.

3. PROCEDIMIENTO REALIZADO

3.1 Instalación y Configuración de JDK 17

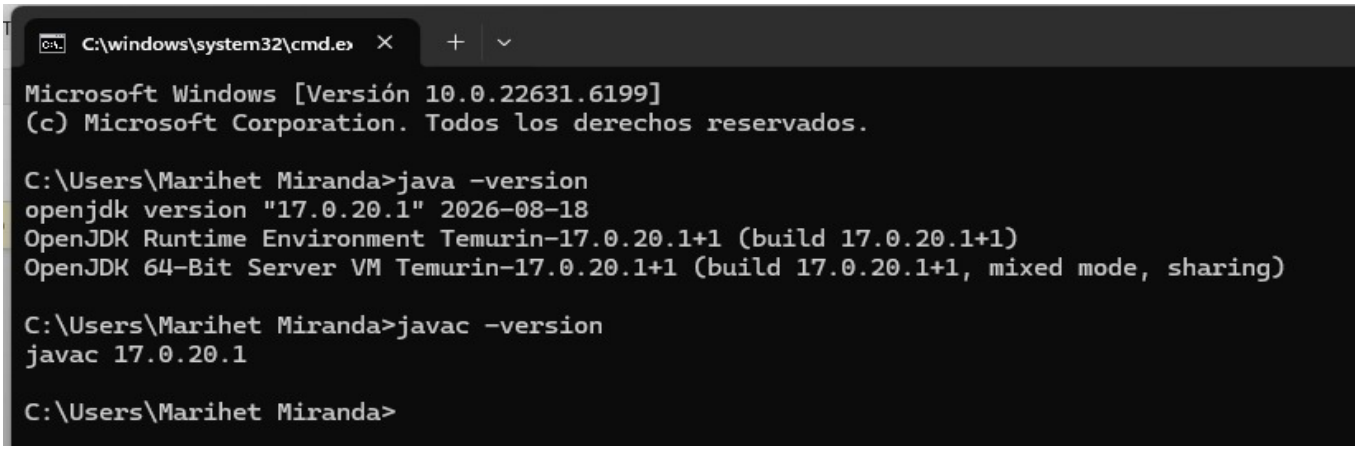
Paso 1: Se descargó el JDK 17 LTS desde el sitio oficial de Eclipse Temurin/Adoptium (adoptium.net)





Paso 2: Se ejecutó el instalador con las opciones por defecto.

Paso 3: Se verificó la instalación abriendo una terminal y ejecutando los siguientes comandos:



Paso 4: Se configuraron las variables de entorno:

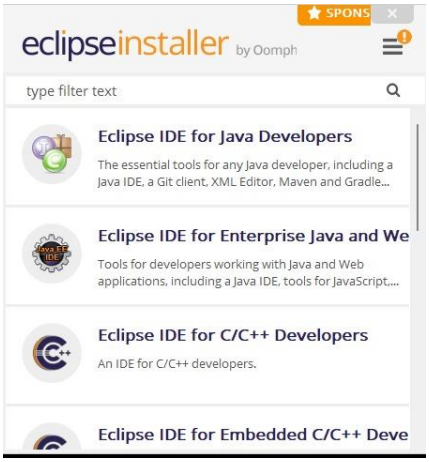
- **Windows:** Se agregó JAVA_HOME y se añadió %JAVA_HOME%\bin al Path.
- **macOS/Linux:** Se agregó la exportación correspondiente en el archivo de perfil de la shell (.bashrc, .zshrc, etc.).

3.2 Instalación de Eclipse IDE

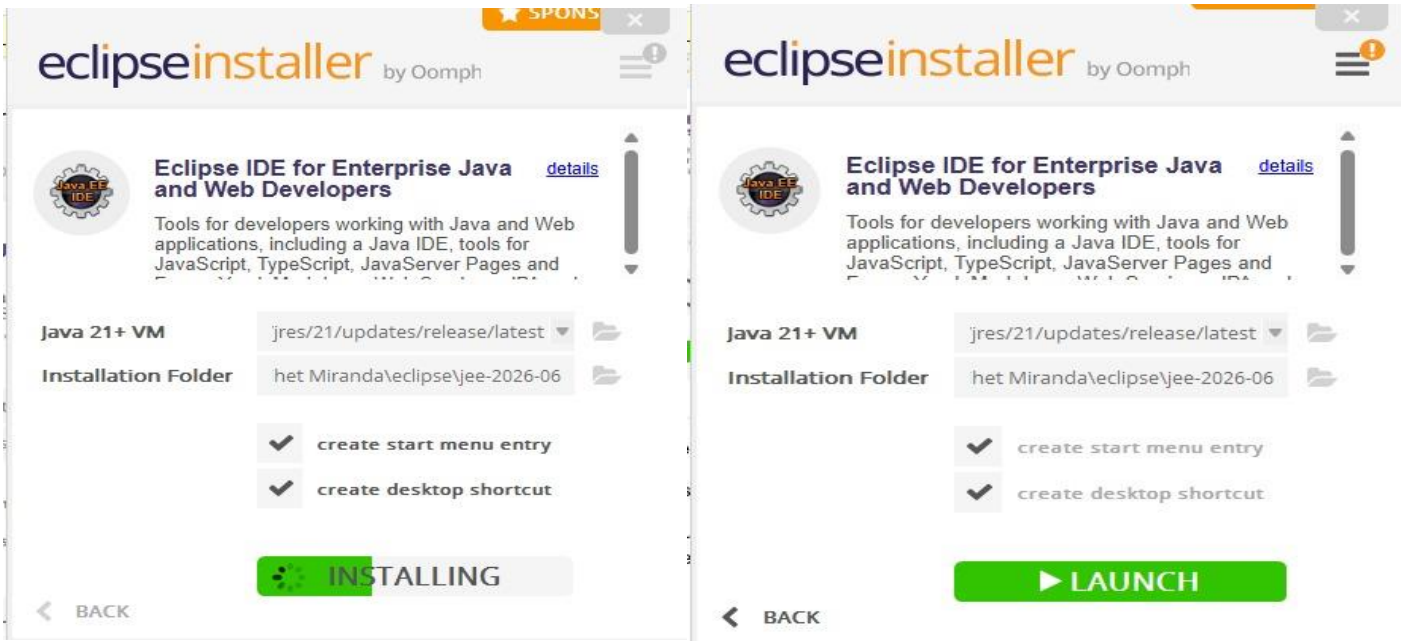
Paso 5: Se descargó el instalador "Eclipse Installer" desde el sitio oficial (eclipse.org/downloads).

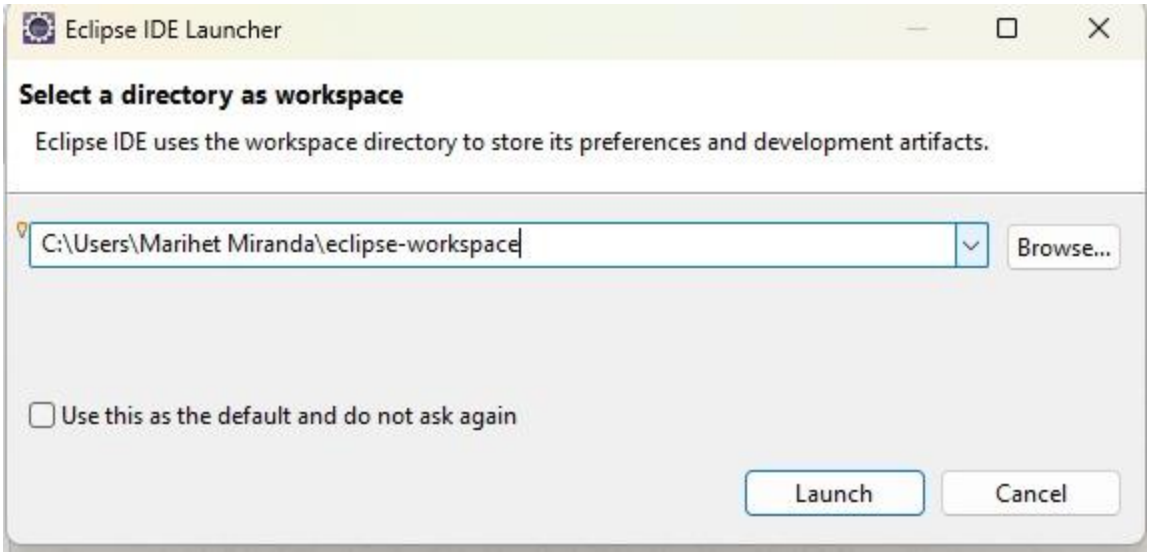


Paso 6: En el instalador, se seleccionó el paquete **"Eclipse IDE for Enterprise Java and Web Developers"** (incluye soporte para Maven y servidores de aplicaciones que se usarán más adelante en el curso).



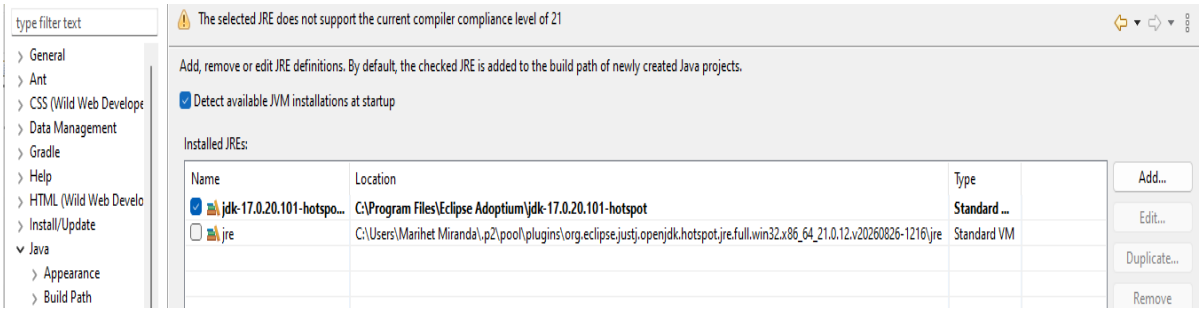
Paso 7: Se completó la instalación y se abrió Eclipse por primera vez, seleccionando el workspace (espacio de trabajo) donde se guardarán los proyectos del semestre.





Paso 8: Se verificó que Eclipse reconozca el JDK 17 instalado:

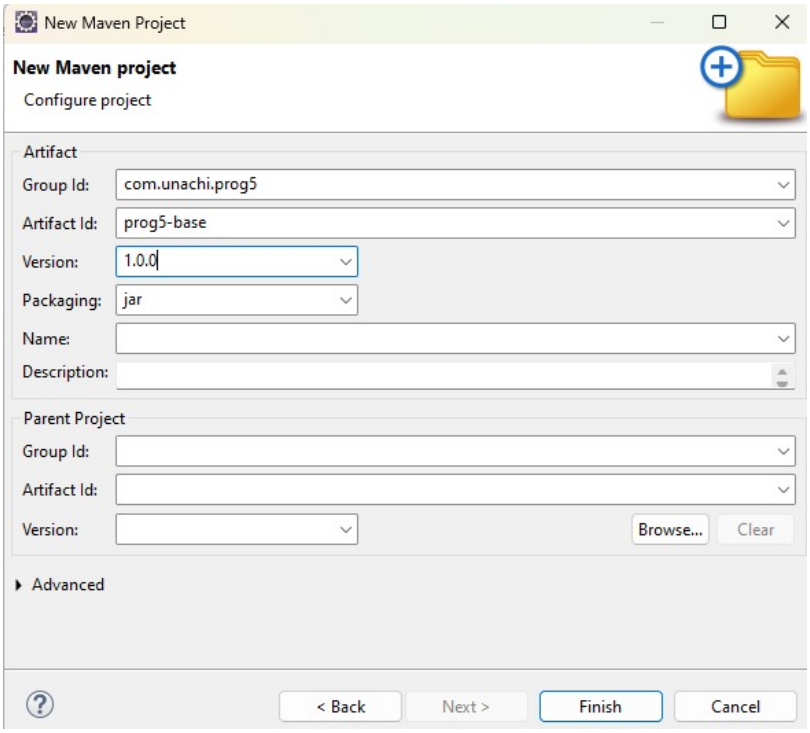
- Window > Preferences > Java > Installed JREs
- Se confirmó que aparece el JDK 17 (si no aparecía, se agregó manualmente con Add… > Standard VM y se ubicó la carpeta de instalación del JDK).



3.3 Creación del Proyecto Maven con Estructura MVC

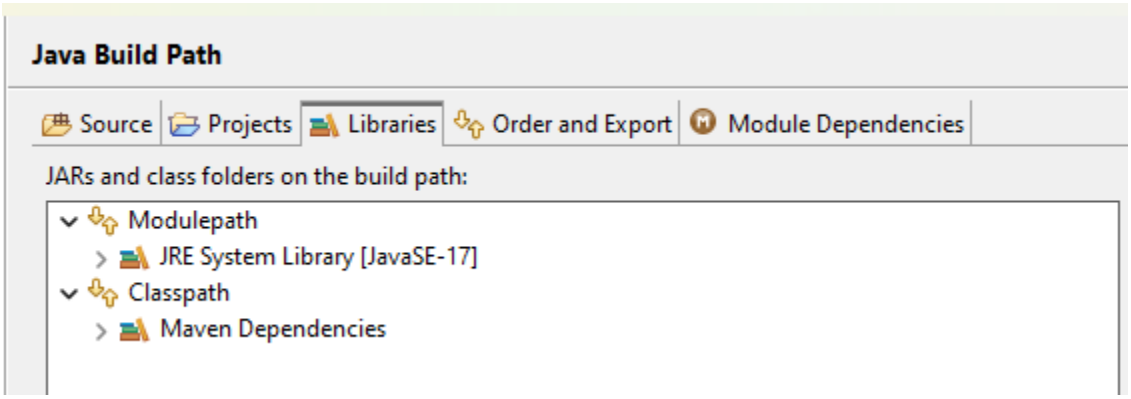
Paso 9: En Eclipse, se creó el proyecto Maven:

- File > New > Maven Project
- Se marcó "Create a simple project"
- Se definió:
    - **Group Id:** com.unachi.prog5
    - **Artifact Id:** prog5-base
    - **Version:** 1.0.0



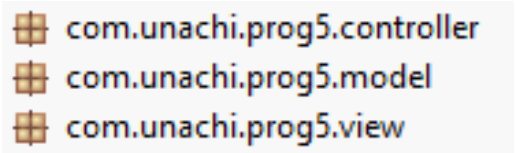
Paso 10: Se verificó que el proyecto use JDK 17:

- Clic derecho sobre el proyecto > Properties > Java Build Path > pestaña Libraries
- Se confirmó que la JRE System Library sea JavaSE-17



Paso 11: Se crearon los paquetes dentro de src/main/java:

- com.unachi.prog5.model
- com.unachi.prog5.view
- com.unachi.prog5.controller



3.4 Creación de las Clases del Proyecto

Se crearon las siguientes clases con el código proporcionado en la guía del laboratorio:

3.4.1 Student.java (Modelo)

```java
package com.unachi.prog5.model;

public class Student {

        private final String id;
        private final String name;
        public Student(String id, String name) {
        this.id = id;
        this.name = name;
        }
        public String summary() {
        return "Student{id='" + id + "', name='" + name + "'}";
        }
       }
```

3.4.2 ConsoleView.java (Vista)

```java
package com.unachi.prog5.view;

public class ConsoleView {
    public void showMessage(String msg) {
        System.out.println(msg);
    }
}

```

3.4.3 AppController.java (Controlador)

```java
package com.unachi.prog5.controller;

import com.unachi.prog5.model.Student;

public class AppController {

    private final ConsoleView view;
    public AppController(ConsoleView view) {
    this.view = view;
    }
    public void run() {
    Student s = new Student("2026-001", "Daniela Pitti");
    view.showMessage("Hola Programacion V");
    view.showMessage("Modelo: " + s.summary());
    }
   }

```

3.4.4 Main.java

```java
package com.unachi.prog5;

import com.unachi.prog5.controller.AppController;

public class main {

    public static void main(String[] args) {
         AppController controller = new AppController(new ConsoleView());
         controller.run();
         }

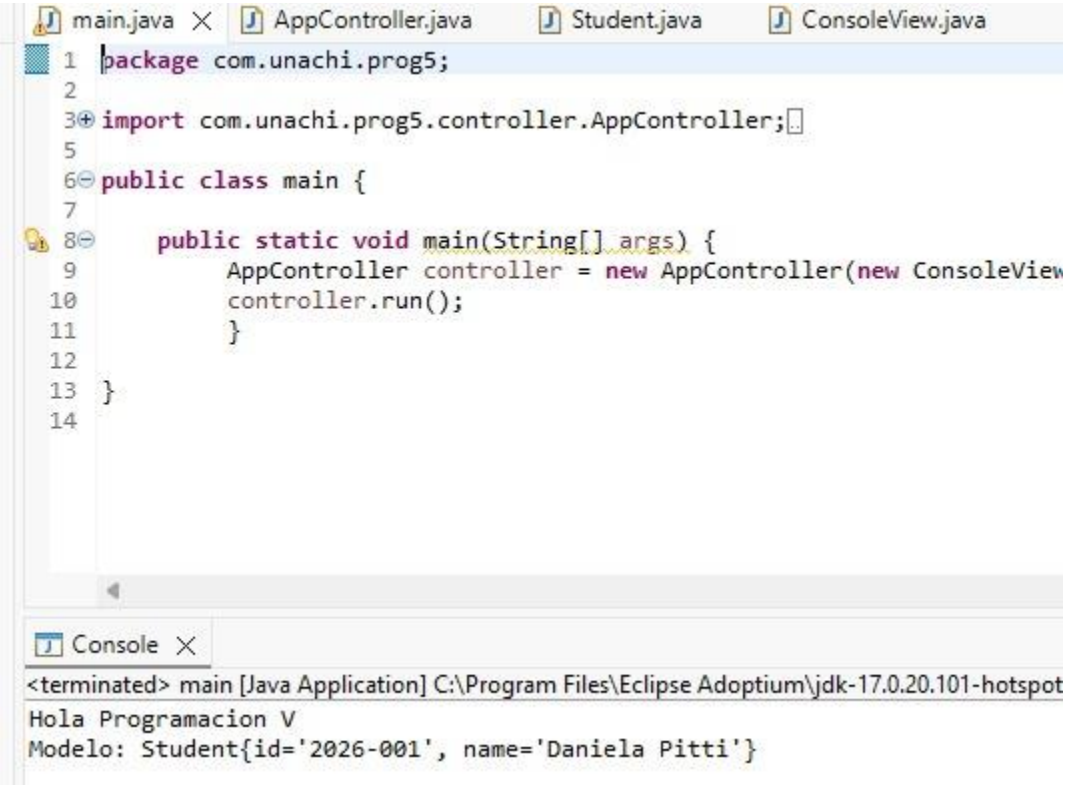
}

```

Paso 12: Se guardaron todos los archivos y se ejecutó Main.java:

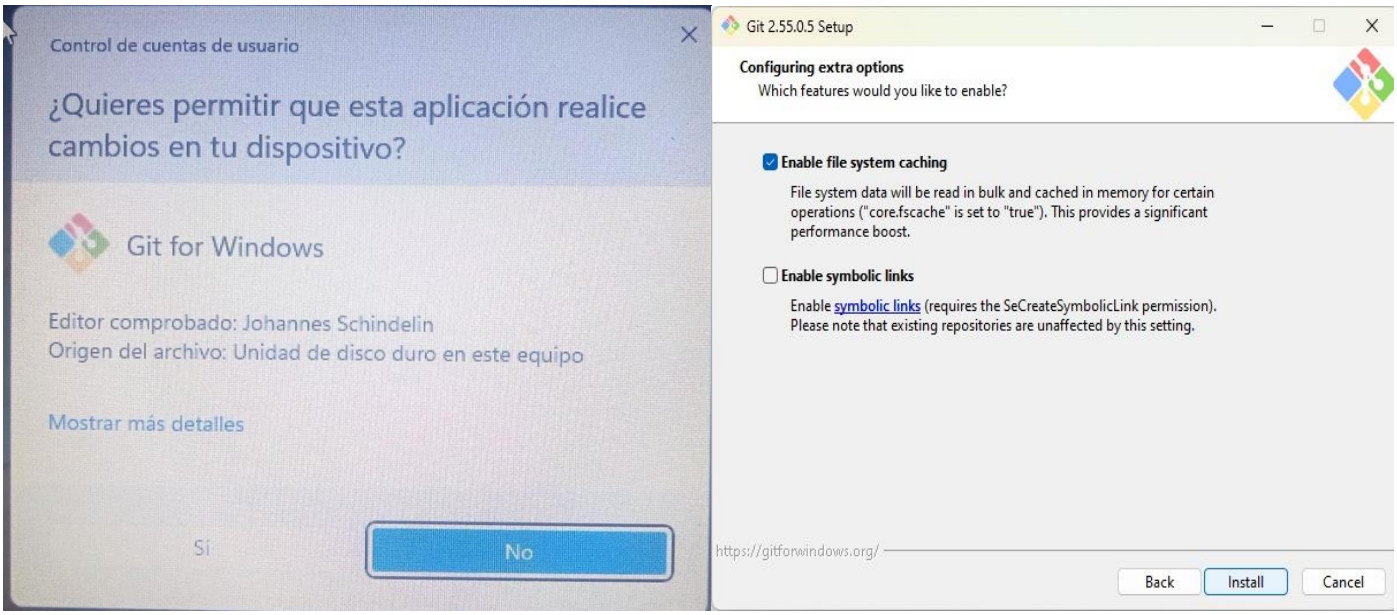
- Clic derecho sobre Main.java > Run As > Java Application

Salida obtenida en la consola de Eclipse:

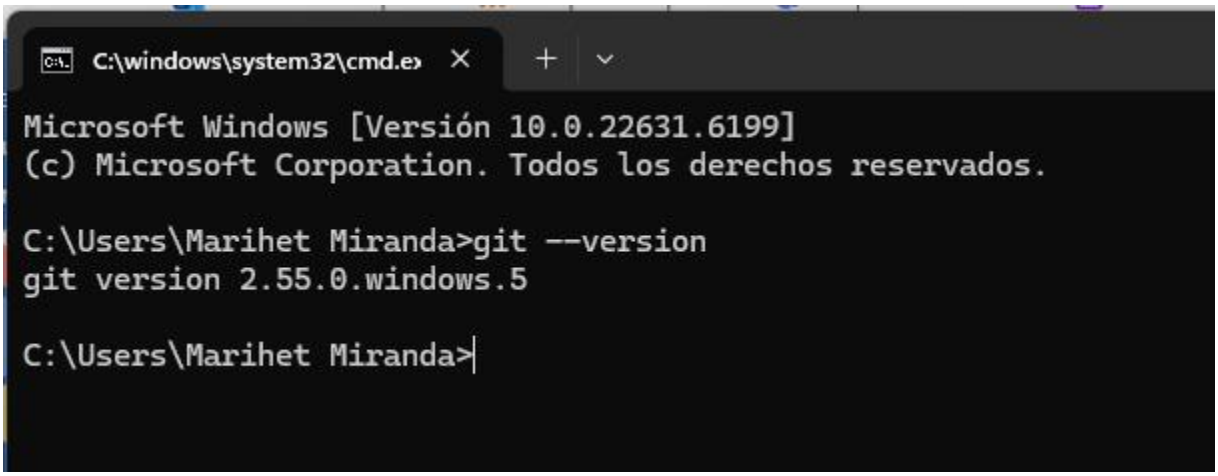


3.6 Instalación y Configuración de Git

Paso 13: Se descargó e instaló Git desde el sitio oficial (git-scm.com). En Windows, se dejaron marcadas las opciones por defecto del instalador (incluye Git Bash).



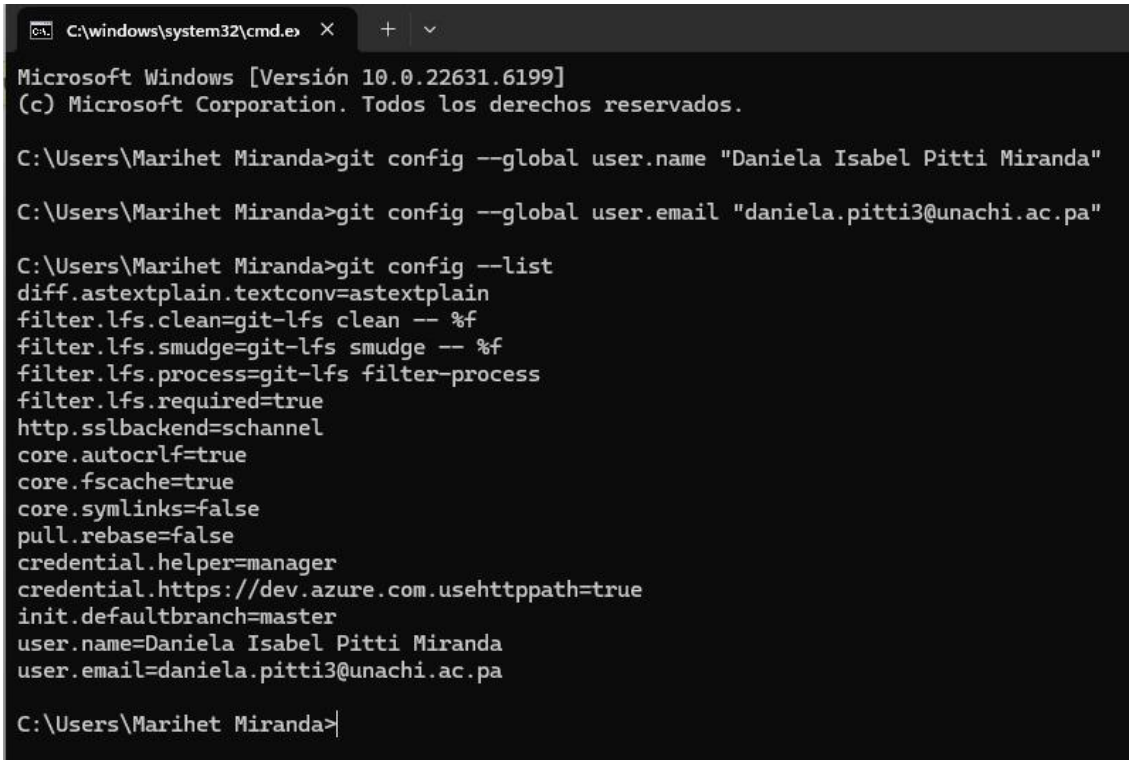
Paso 14: Se abrió una terminal y se verificó la instalación:



Paso 15: Se configuró el nombre de usuario:

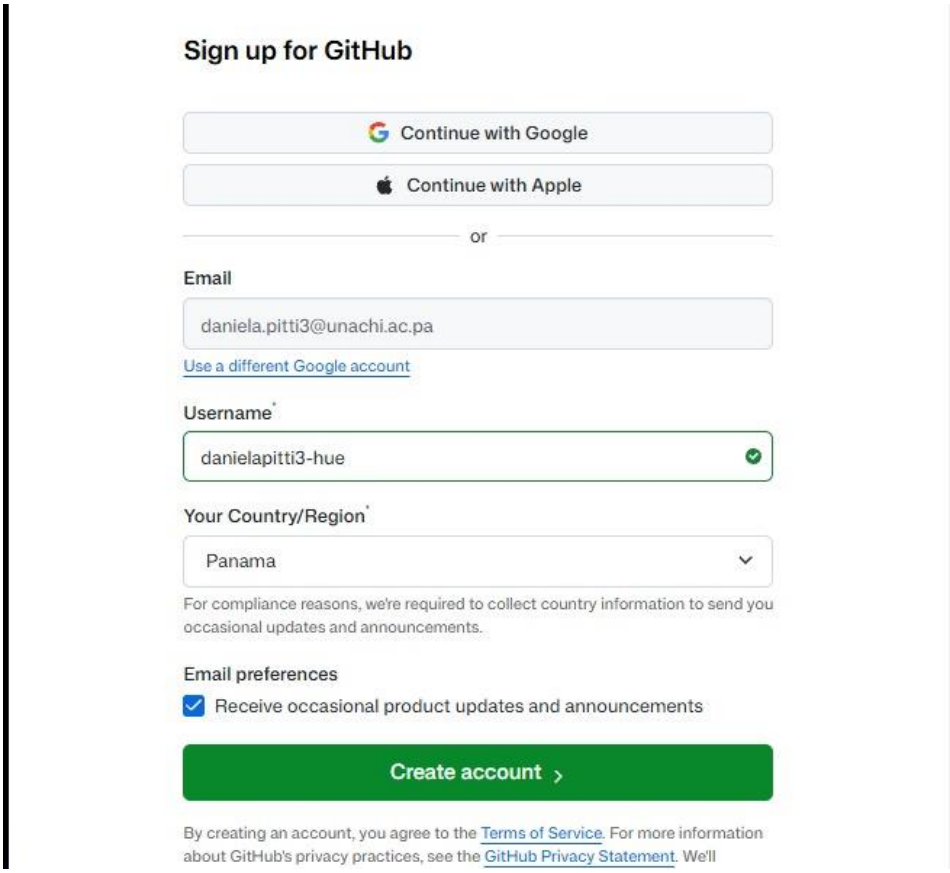
Paso 16: Se configuró el correo electrónico:

Paso 17: Se verificó la configuración:



3.7 Creación de Cuenta en GitHub

Paso 18: Se creó una cuenta en GitHub (github.com) utilizando el correo institucional.

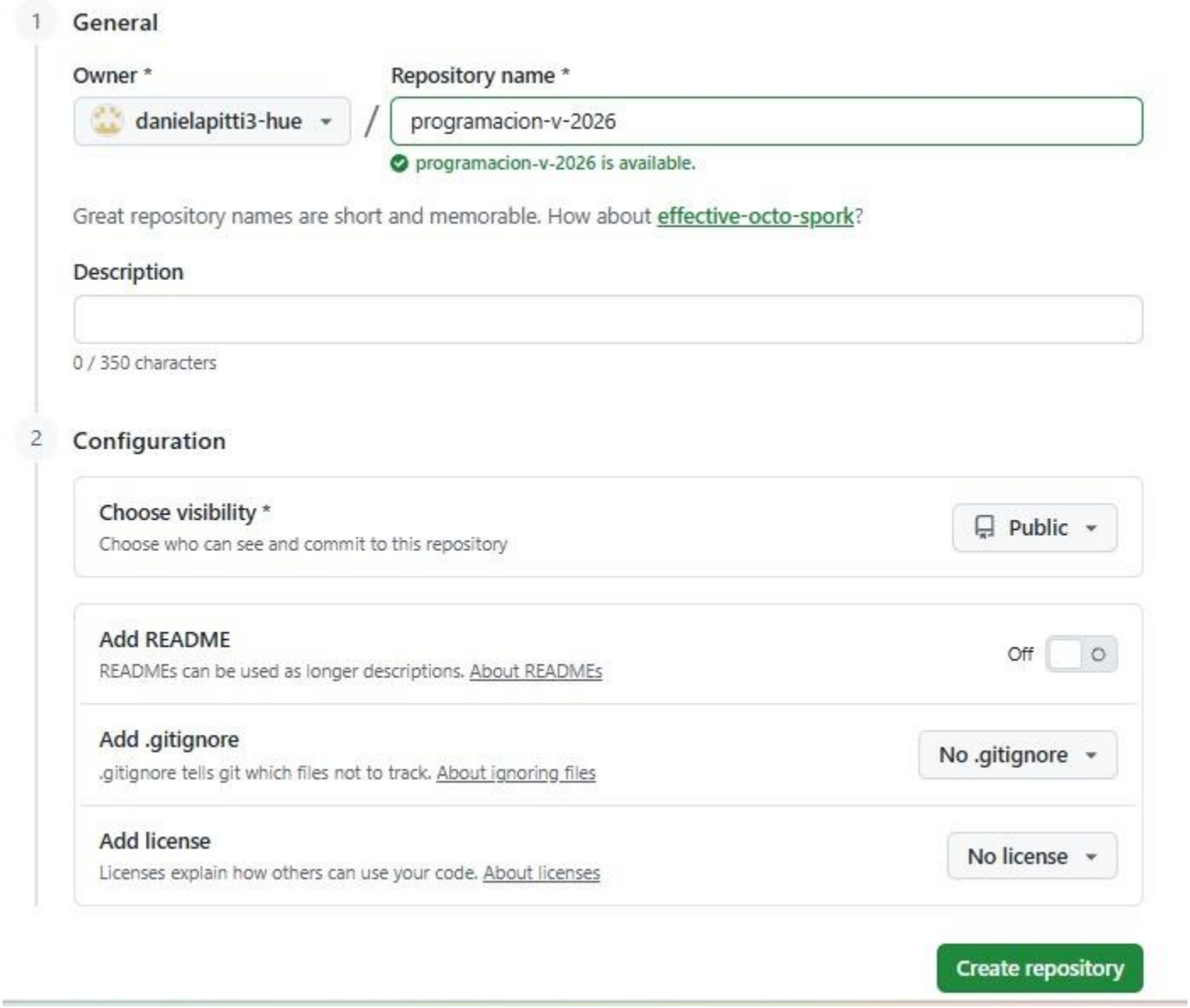


3.8 Creación del Repositorio Remoto en GitHub

Paso 19: Se inició sesión en GitHub y se hizo clic en el botón New (o el símbolo + > New repository).

Paso 20: Se nombró el repositorio programacion-v-2026 y se seleccionó Public o Private según lo indicado por el docente.

Paso 21: IMPORTANTE: No se marcó la opción "Add a README file" ni ninguna otra opción de inicialización, para evitar conflictos al conectar el repositorio local.



Paso 22: Se copió la URL de HTTPS del repositorio:

https://github.com/danielapitti3-hue/programacion-v-2026

3.9 Inicialización del Repositorio Local

Paso 23: Se abrió una terminal en la carpeta de trabajo ProgramacionV_2026-II (también se puede abrir en Eclipse con clic derecho sobre el proyecto > Show In > Terminal).

Paso 24: Se inicializó Git en la carpeta:

```
C:\windows\system32\cmd.exe
Microsoft Windows [Versión 10.0.22631.6199]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git init
Initialized empty Git repository in C:/Users/Marihet Miranda/eclipse-workspace/prog5-base/.git/

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>
```

Paso 25: Se verificó el estado del repositorio:

```
C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git status
On branch master

No commits yet

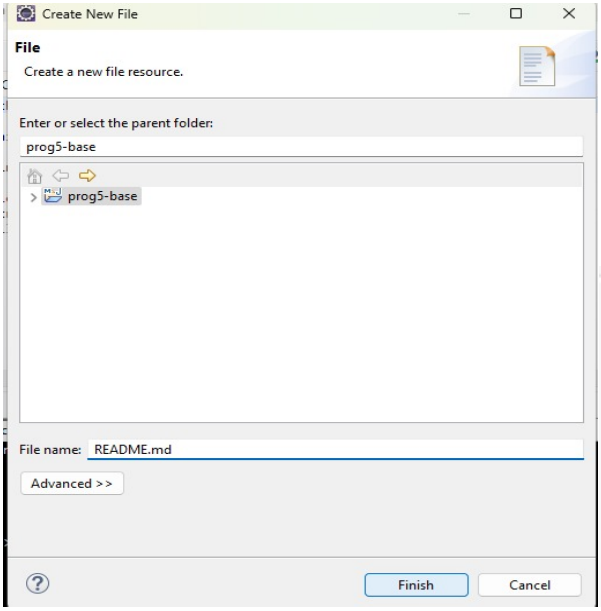
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .classpath
        .project
        .settings/
        pom.xml
        src/
        target/

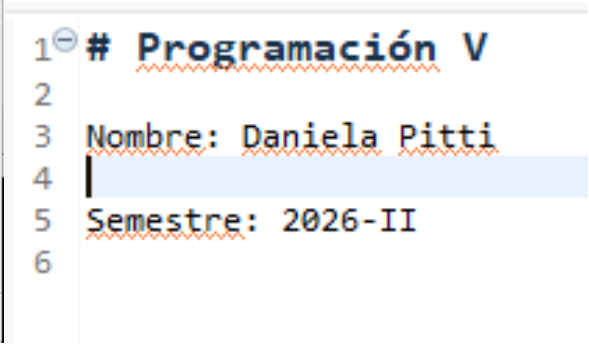
nothing added to commit but untracked files present (use "git add" to track)

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>
```

3.10 Creación del Primer Commit

Paso 26: Se creó el archivo README.md en la carpeta del proyecto con el siguiente contenido:

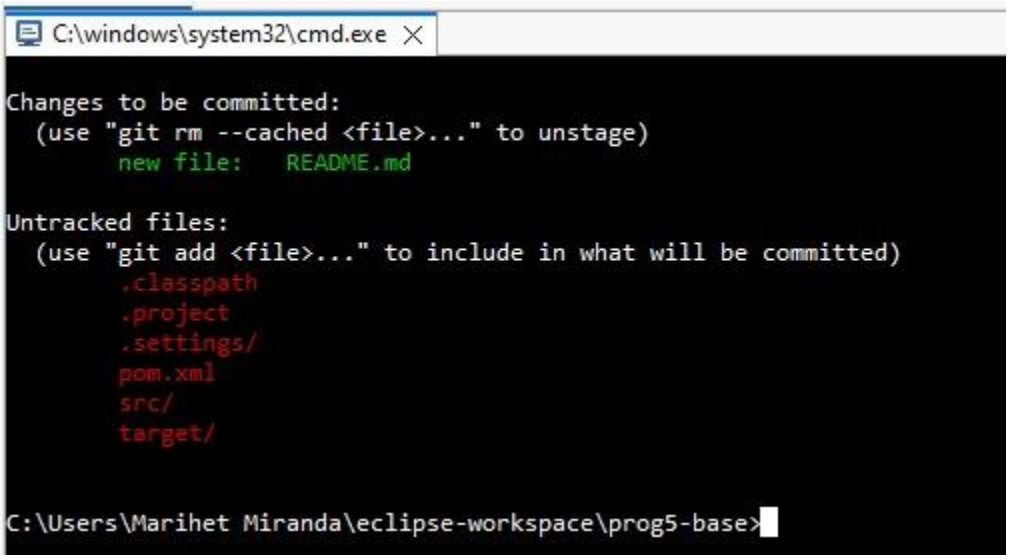




Paso 27: Se agregaron los archivos al área de preparación (staging)

git add .

Paso 28: Se verificó qué quedó preparado para el commit:



Paso 29: Se realizó el primer commit:



3.11 Conexión con el Repositorio Remoto y Push

Paso 30: Se conectó el repositorio local con el repositorio remoto de GitHub:





Paso 31: Se verificó que el remoto quedó correctamente conectado:

```
C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git remote -v
origin  https://github.com/danielapitti3-hue/programacion-v-2026.git (fetch)
origin  https://github.com/danielapitti3-hue/programacion-v-2026.git (push)

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>
```

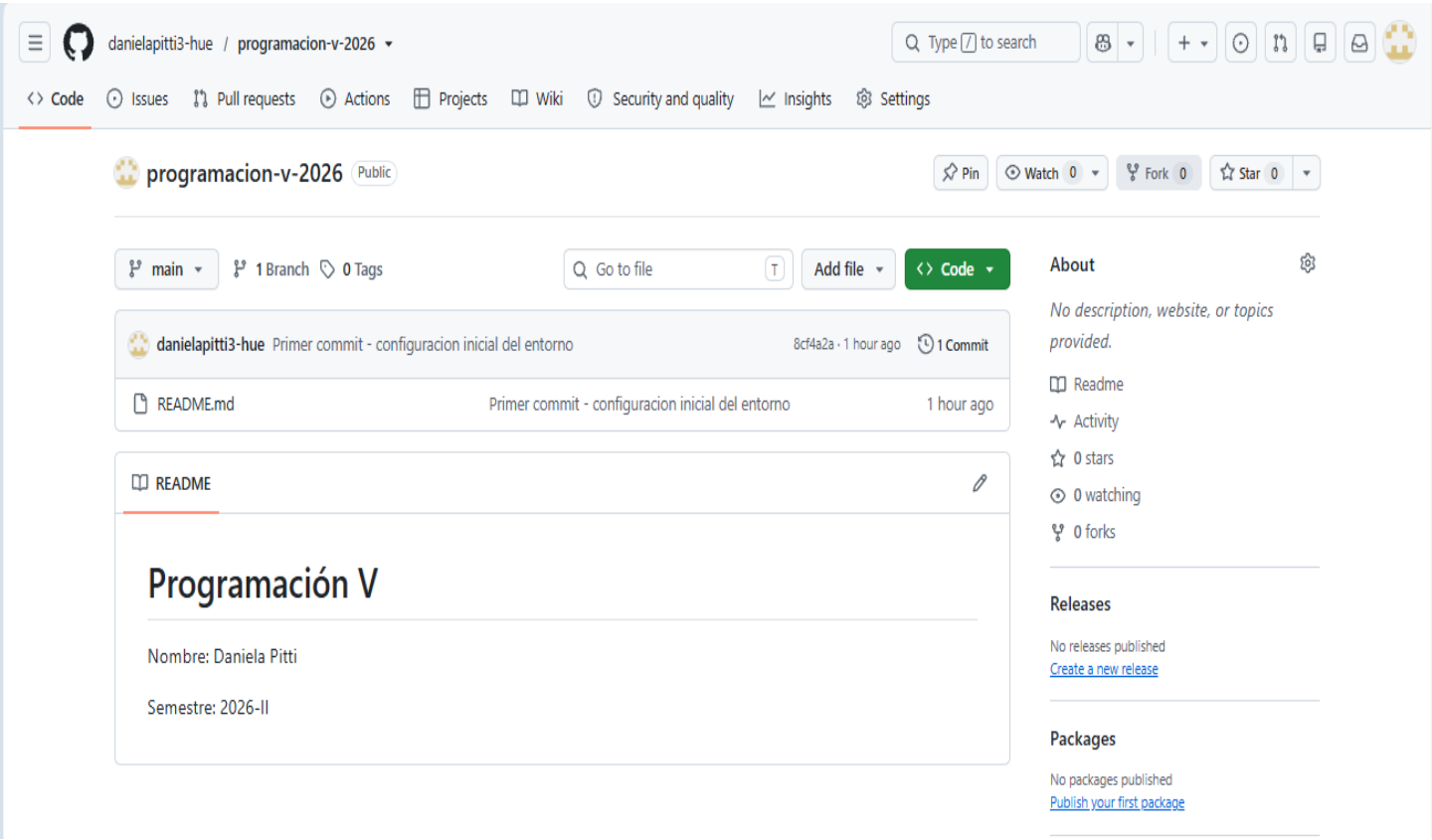
Paso 32: Se aseguró que la rama principal se llame main

git branch -M main

Paso 33: Se subió el commit al repositorio remoto

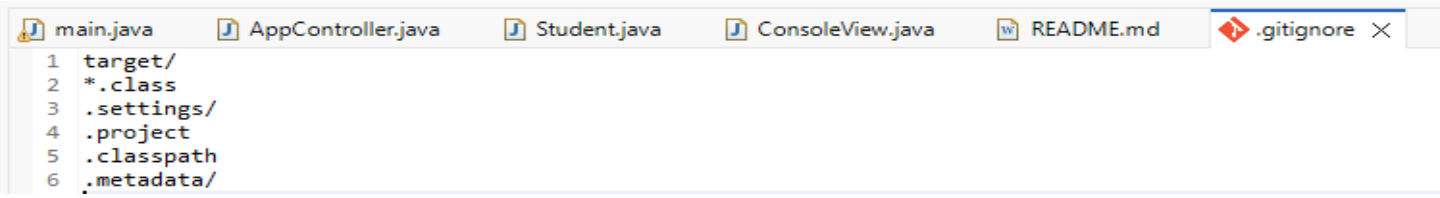
git push -u origin main

Paso 34: Se recargó la página del repositorio en GitHub y se verificó que el archivo README.md y el commit realizado aparezcan reflejados.



3.12 Configuración del Archivo .gitignore

Paso 35: Se creó un archivo .gitignore en la raíz del proyecto con el siguiente contenido para evitar subir archivos generados automáticamente por Eclipse y Maven:



Paso 36: Se agregó, se realizó commit y push:

```
branch 'main' set up to track 'origin/main'.

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git add .gitignore

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        pom.xml
        src/


C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>
```

```
C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git commit -m "Agregar gitignore para Java Maven Eclipse"
[main e1a288d] Agregar gitignore para Java Maven Eclipse
 1 file changed, 6 insertions(+)
 create mode 100644 .gitignore

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 373 bytes | 124.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/danielapitti3-hue/programacion-v-2026.git
   8cf4a2a..e1a288d  main -> main

C:\Users\Marihet Miranda\eclipse-workspace\prog5-base>
```

3.13 Reconocimiento Visual del Patrón de Capas

Aplicación seleccionada: Instagram

Paso 37: Identificación de la Capa de Presentación:

- Pantalla principal con el feed de publicaciones
- Botones de like, comentar, compartir
- Barra de navegación (Inicio, Búsqueda, Reels, Perfil)
- Perfil de usuario (foto, biografía, publicaciones)
- Historias en la parte superior

Paso 38: Identificación de la Capa de Negocio:

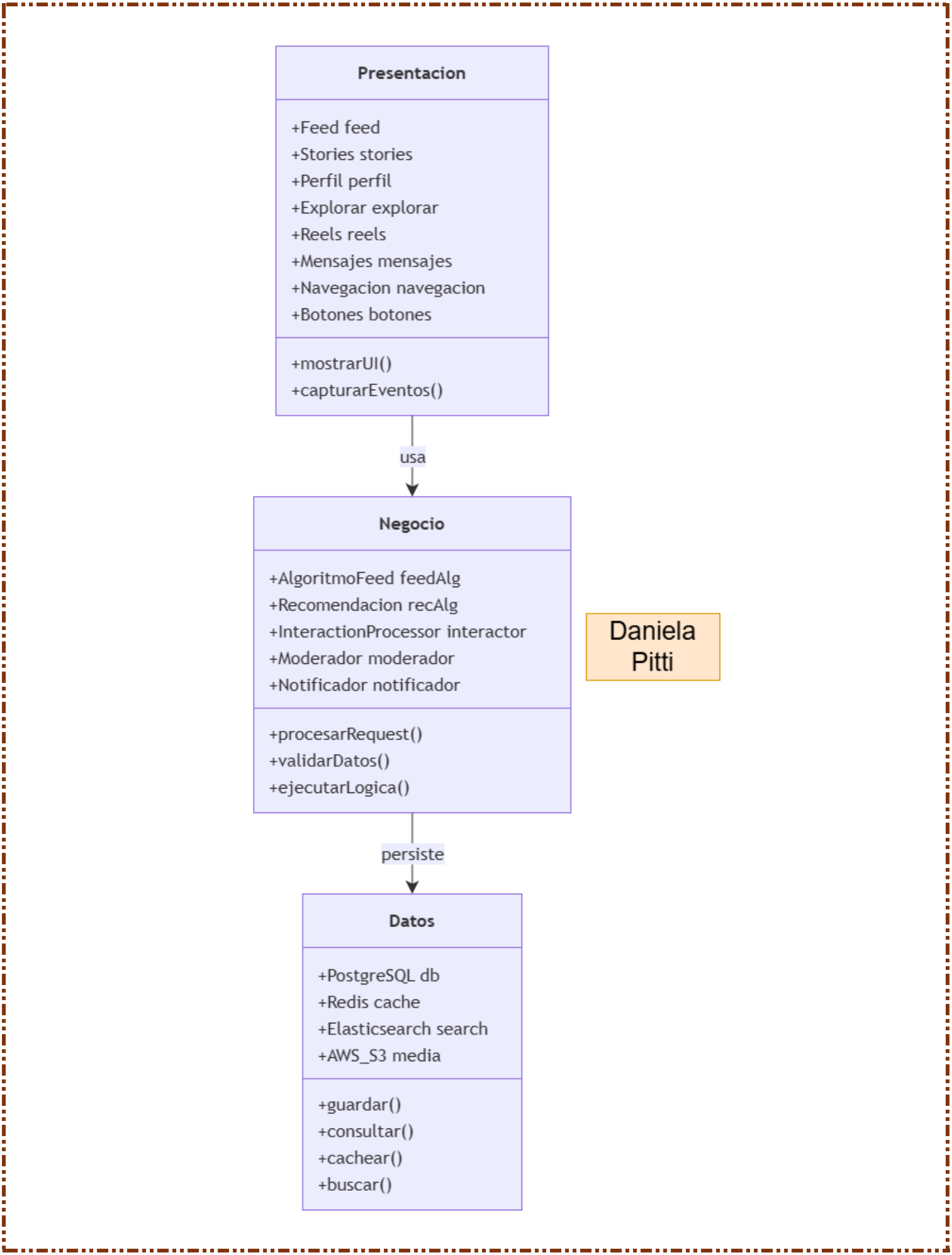
- Algoritmo de clasificación del feed personalizado
- Sistema de recomendación de cuentas para seguir
- Procesamiento de likes y comentarios
- Validación de contenido y spam
- Sistema de notificaciones en tiempo real
- Algoritmo de Explorar (descubrimiento de contenido)

Paso 39: Identificación de la Capa de Datos:

- Base de datos de usuarios (nombre, email, contraseña hash, fecha registro)
- Base de datos de publicaciones (imagen, video, texto, fecha, ubicación)
- Base de datos de interacciones (likes, comentarios, guardados)
- Base de datos de relaciones (seguidores, seguidos)
- Datos de historias temporales (24 horas)
- Mensajes directos y conversaciones

3.14 Elaboración del Diagrama de Capas

Paso 40: Se utilizó la herramienta de diagramación en línea **draiw.io** para elaborar el diagrama de tres capas que representa la aplicación Instagram.


Presentacion
+Feed feed
+Stories stories
+Perfil perfil
+Explorar explorar
+Reels reels
+Mensajes mensajes
+Navegacion navegacion
+Botones botones
+mostrarUI()
+capturarEventos()
usa
Negocio
+AlgoritmoFeed feedAlg
+Recomendacion recAlg
+InteractionProcessor interactor
+Moderador moderador
+Notificador notificador
+procesarRequest()
+validarDatos()
+ejecutarLogica()
persiste
Datos
+PostgreSQL db
+Redis cache
+Elasticsearch search
+AWS_S3 media
+guardar()
+consultar()
+cachear()
+buscar()
Daniela Pitti

6. PREGUNTAS DE ANÁLISIS

Pregunta 1:

¿Por qué es importante llevar un control de versiones (Git) desde el primer proyecto del semestre y no solo en proyectos grandes?

Es importante porque:

- **Tienes historial completo**: Cada cambio queda registrado. Es como un diario de tu proyecto donde ves cómo evolucionó y qué decisiones tomaste.
- **Puedes equivocarte sin miedo**: Si algo sale mal, puedes volver atrás sin perder horas de trabajo. Es como un "Ctrl+Z" pero a nivel de proyecto entero.
- **Aprendes haciendo**: Cometer errores en un proyecto pequeño no duele tanto. Es el momento perfecto para aprender a usar ramas, merges y resolver conflictos, sin la presión de un proyecto gigante.
- **Te preparas para el mundo real**: En el trabajo, nadie desarrolla sin control de versiones. Si ya lo usas desde la universidad, cuando llegues a una empresa no te sentirás perdido.
- **Construyes tu portafolio**: Tener tus proyectos en GitHub es tu carta de presentación. Los empleadores pueden ver cómo programas, cómo organizas tu código y cómo evolucionas.
- **Creas disciplina**: Hacer commits pequeños con mensajes claros es un hábito que te diferencia como profesional.

Pregunta 2:

En la aplicación que eligió para el diagrama de capas (Instagram), ¿qué problema podría ocurrir si la capa de presentación accediera directamente a la base de datos, sin pasar por la capa de negocio?

Si la capa de presentación accediera directo a la base de datos:

- **Inseguridad**: Cualquiera podría acceder a perfiles privados, mensajes o fotos sin autorización. Sería como tener las puertas abiertas.
- **Caos en las reglas**: Podrías dar likes infinitos, borrar publicaciones ajenas o comentar sin control. Sería el caos total.
- **Rendimiento horrible**: La app haría consultas mal optimizadas, saturando la base de datos. Instagram colapsaría con muchos usuarios.
- **Mantenimiento imposible**: Si cambias la base de datos, tienes que rehacer toda la interfaz. Un dolor de cabeza enorme.
- **Datos inconsistentes**: Tendrías comentarios sin dueño, likes sin usuario… pura basura en la base de datos.
- **Adiós a la escalabilidad**: No podrías crecer sin romper todo.

Pregunta 3:

¿Qué ventajas ofrece trabajar con un repositorio remoto (como GitHub) en lugar de guardar el código únicamente en la computadora local?

Las ventajas son:

- **Tus archivos están seguros**: Si tu computadora explota, tu código sigue en la nube. Es tu respaldo automático.
- **Trabajas desde cualquier lado**: En casa, en la universidad o en un café. Tu código te sigue a todas partes.
- **Trabajo en equipo sin pisarte**: Varias personas pueden trabajar al mismo tiempo, cada quien en su rama. Como tener carriles separados.
- **Todo queda registrado**: Cada cambio queda documentado. Sabes quién, cuándo y por qué se hizo cada cosa.
- **Revisión de código**: Los Pull Requests son como un filtro de calidad. Otros revisan tu código antes de que entre al proyecto.
- **Automatización**: Puedes configurar pruebas automáticas cada que subes código. Como tener un asistente que revisa todo.
- **Tu vitrina profesional**: GitHub es tu portafolio. Los reclutadores ven tu código y tu evolución.
- **Gestión de tareas**: Puedes crear issues para trackear bugs y pendientes.

Pregunta 4:

Investigue: ¿qué diferencia existe entre un IDE y un simple editor de texto como Notepad++ o Sublime Text?

La diferencia es como entre un taller mecánico completo y una simple caja de herramientas. Ambos sirven, pero uno te lo pone mucho más fácil:

Aspecto	*IDE (como Eclipse)*	*Editor de Texto (Notepad++/Sublime)*
Compilar	Lo hace solo, con un botón	Tienes que abrir la terminal y compilar manualmente
Depurar	Tiene herramientas para pausar el código, ver variables paso a paso	Nada, solo puedes poner prints y rezar que funcione
Autocompletado	Te sugiere métodos, variables, hasta te completa imports	Solo autocompleta palabras que ya escribiste antes
Organizar proyectos	Maneja estructuras complejas con Maven, carpetas, dependencias	Solo ves archivos sueltos en una lista
Refactorizar	Puedes renombrar una clase y automáticamente actualiza todo el código	Tienes que hacer buscar y reemplazar manualmente
Git	Tiene integración visual, ves los cambios, haces commits desde la misma interfaz	Necesitas instalar plugins o usar la terminal
Ejecutar pruebas	Corre pruebas unitarias con solo un click	Tienes que ejecutarlas desde la terminal
Extensiones	Tiene un montón de plugins para casi todo	Hay extensiones, pero más limitadas
Gestión de dependencias	Maneja Maven y Gradle de forma integrada	No, tienes que hacerlo manualmente
Cuándo usarlo	Para proyectos grandes y complejos	Para editar archivos rápidos, configuraciones, scripts

En resumen, el IDE es para cuando realmente necesitas potencia y herramientas profesionales. Un editor de texto está bien para cambiar una línea en un archivo de configuración, pero para desarrollar una aplicación completa, el IDE te ahorra horas de trabajo.

7. CONCLUSIONES

Al finalizar este laboratorio, logré:

- **Configurar el entorno de desarrollo** instalando JDK 17 y Eclipse, asegurando que todo funcione bien para el semestre. Esto no fue nada fácil, tuve varios problemas con las versiones y la compatibilidad, y la verdad me dolía la cabeza de tanto intentarlo.
- **Crear un proyecto Maven con MVC**, entendiendo la separación entre modelo, vista y controlador. Esto es la base del patrón de capas que veremos después. Me costó bastante entender cómo se conectaban las partes, pero al final le fui agarrando el hilo.
- **Implementar Git**, haciendo mi primer commit y subiéndolo a GitHub, entendiendo que versionar desde el principio es clave sin importar el tamaño del proyecto. La parte de subirlo al repositorio remoto fue un dolor de cabeza, no sabía bien cómo se hacía y me salían errores que no entendía.
- **Reconocer el patrón de capas en Instagram**, identificando presentación, negocio y datos en una aplicación real. Esto fue lo más llevadero, aunque también me costó entender cómo se separaban las responsabilidades.
- **Diferenciar IDE de editor de texto**, entendiendo las ventajas que ofrecen las herramientas integradas.

La verdad, este laboratorio me costó mucho trabajo. Hubo momentos en los que pensé que nunca iba a funcionar, especialmente cuando trataba de importar los proyectos o cuando el repositorio remoto no quería sincronizarse. Algunos documentos tardaban tanto en cargar que llegué a pensar que se había quedado pegado todo y que iba a tener que empezar de nuevo.

Pero bueno, al final todo salió. Aprendí que en la programación las cosas no siempre salen a la primera, y que hay que tener paciencia y seguir intentándolo. Esto me sirvió para darme cuenta de que la práctica es la única forma de ir mejorando, y que los errores son parte del proceso.

Este laboratorio sentó las bases para los próximos temas, donde profundizaremos en arquitectura de software. Tener todo configurado desde ahora evita pérdidas de tiempo y nos permite concentrarnos en lo importante: entender cómo se construyen aplicaciones bien organizadas.

8. REFERENCIAS

- Huaman, W. C. (2018). *Los 10 patrones comunes de arquitectura de software*. Medium.
- Blancarte, O. (2020). *Introducción a la arquitectura de software*.
- Documentación oficial de Git: https://git-scm.com/doc
- Documentación oficial de Eclipse IDE: https://www.eclipse.org/documentation
- Eclipse Temurin (Adoptium) — distribución del JDK 17: https://adoptium.net
- Documentación oficial de draw.io: https://www.diagrams.net/